

Chapter 5 (PART 1)

Structural Model

OBJECT ใน Object-oriented Development

หลักการของวัตถุ

- ▶ -Object เป็นกลุ่มก้อนของ Entity ที่ประกอบด้วย attribute หรือ คุณสมบัติ, พฤติกรรม หรือ method และ อาจมีสถานะด้วย
- ▶ การพัฒนาเชิงวัตถุ มีมุมมองต่อปัญหาเป็น object ประกอบกับการเขียนโปรแกรมเชิงวัตถุ
- ▶ แบ่งปัญหา เป็น object ซึ่งมีความผูกพันกันน้อย (Loose Couple) ทำให้การแก้ไข และดูแลส่วนต่างๆของระบบ หรือ object ทำได้ง่าย และลดผลกระทบต่อส่วนอื่นของระบบ

ตัวอย่าง Object “ Customer ”

Attribute	Method	State	Identity	Responsibility
Email	Create Order	-Not order	Email	-Enter order
Name	Search order	-Ordered		- Paid
Surname	Search available-item	-Paid		
Card-id	Pay	-not paid		
Mobile-number	Receive item			
address	search payment status			

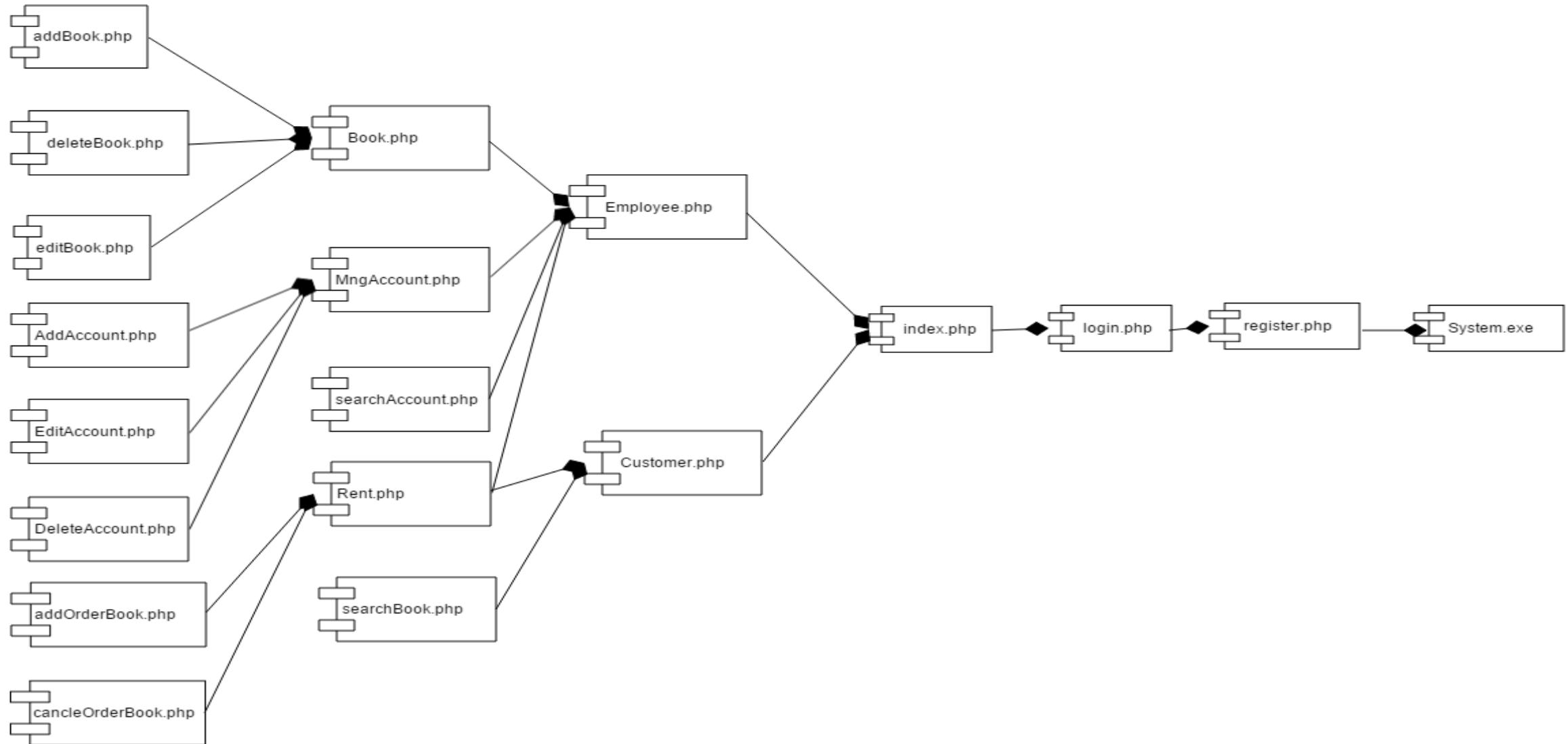
Structured Diagram ได้แก่

- ▶ **-Object diagram** : แสดง Object และ relationship ระหว่าง object
- ▶ **-Class diagram** : แสดงโครงสร้างทาง ตรรกะ ของระบบ

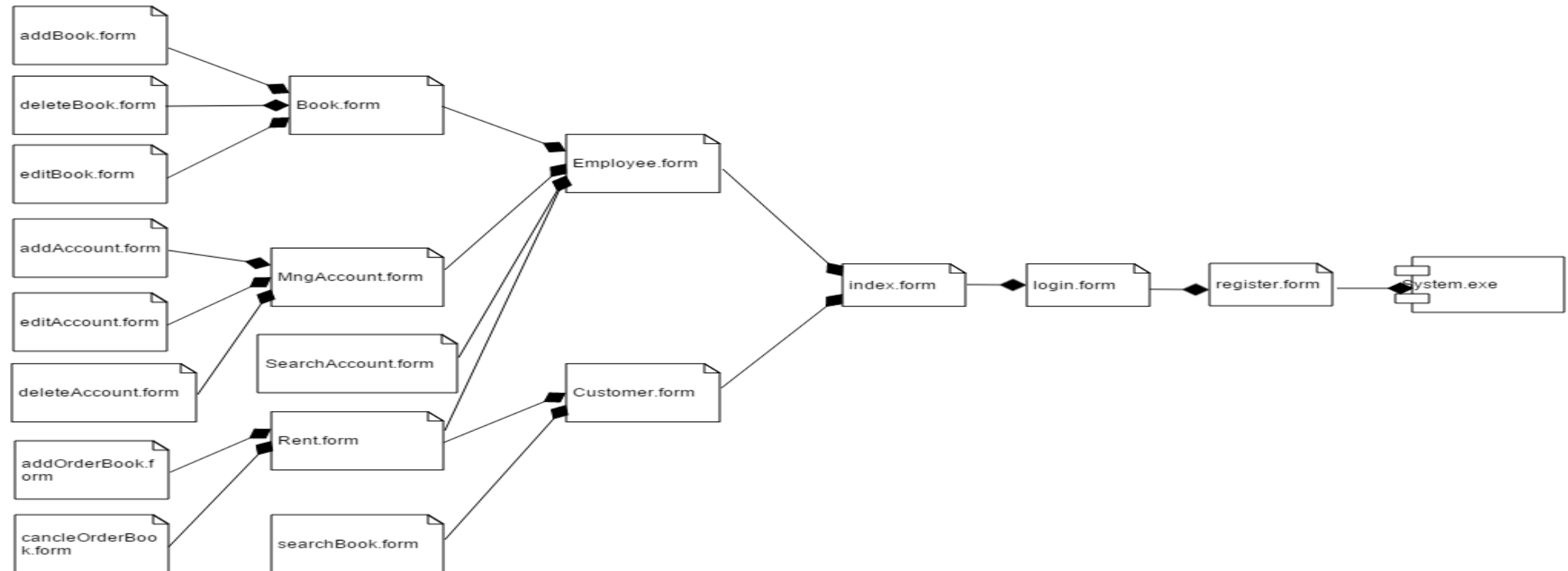
ใช้ออกแบบ Database

- ▶ **-Component diagram** : แสดงโครงสร้างทางกายภาพของ Software
 - **Component Diagram ประเภท Working Logic Subsystem** :
แสดงโครงสร้างและความสัมพันธ์ขององค์ประกอบของโปรแกรม
 - **Component Diagram ประเภท Presentation Logic Subsystem**
แสดงโครงสร้างและความสัมพันธ์ขององค์ประกอบของ User Interface

ตัวอย่าง Working Logic Subsystem



ตัวอย่าง Presentation Logic Subsystem



Structured Diagram ได้แก่ (ต่อ)

▶ **-Deployment diagram** : แสดงการติดต่อระหว่าง

Hardware กับ Software

▶ **-Interaction diagram** : แสดงพฤติกรรมของระบบ

แสดงการโต้ตอบ มีปฏิสัมพันธ์ระหว่างองค์ประกอบต่างๆ เช่น

ปฏิสัมพันธ์ระหว่าง Object ในระบบ

Structured Diagrams

1) **Object** คือ วัตถุ ที่มีตัวตน หรือ ไม่มีตัวตน เป็นคำนาม
(noun) ระหว่าง 2 Object ในระบบจะมีความสัมพันธ์
หรือ Relationship

การปฏิสัมพันธ์ระหว่าง 2 Object ขึ้นไป เรียกว่า Interaction

- ▶ Object มี คุณสมบัติ (Attribute)
- ▶ Class เป็น พิมพ์เขียวของ Object

Structured Diagrams

- ▶ ในระบบการลงทะเบียนเรียน นักศึกษา ชื่อ นาย **A**
ลงทะเบียนวิชา **COS3108**
- ▶ OBJECT คือ นักศึกษา นาย **A** และ วิชา COS3108
- ▶ RELATIONSHIP : ความสัมพันธ์ คือ
ลงทะเบียน ระหว่าง นาย **A** กับ วิชา **COS3108**
- ▶ -INTERACTION คือ
นาย **A** <- ลงทะเบียน -> วิชา **COS3108**

องค์ประกอบใน Object-oriented Development (ต่อ)

- ▶ 1) Object : ใน Object มีองค์ประกอบได้แก่
 - **Attribute**
 - **การกระทำ (Method)** กิจกรรมที่สามารถกระทำกับ Object นั้นๆได้
 - **สถานะ (state)** ซึ่งสถานะของ object ณ เวลาหนึ่งๆ เปลี่ยนแปลงได้ เมื่อมีการกระตุ้นจาก object อื่นโดยการส่ง message (ใน JAVA สถานะของ object คือ value of attribute)
 - **สิ่งบ่งชี้ (identity)**
 - **ความรับผิดชอบ (Responsibility)**

2) **DOMAIN** คือ กรอบความสนใจ หรือ เป้าหมายการทำงาน ที่มีต่อ OBJECT

ภายใน 1 DOMAIN มี Object ตั้งแต่ 2 Object ขึ้นไป

ใน 1 OBJECT สามารถอยู่ในหลาย Domain ได้

3) **CONCEPT** คือ ความคิดรวบยอดที่มีให้กับ OBJECT นั้นๆ ภายในกรอบ

4) **CLASS** = กลุ่ม OBJECT ที่ได้จากการให้ CONCEPT กับ Object ในกรอบที่กำหนด
หรือ เป็นกลุ่มของ Object ที่มีโครงสร้างพื้นฐานพฤติกรรมเดียวกัน

= ABSTRACT OBJECT

= TEMPLATE (พิมพ์เขียว) ของ OBJECT ดังนั้น **Class** ไม่สามารถทำงานได้

Encapsulation = ผู้ใช้ไม่สามารถเห็นรายละเอียด (ข้อมูลและฟังก์ชัน) ภายใน เนื่องจากรายละเอียด

ถูกซ่อนและบรรจุไว้ใน Capsule แต่สามารถใช้งาน Object ได้ด้วย Operation หรือ Method

Object ใดที่ต้องการเข้าถึงข้อมูล ใน Object อื่น หรือ Object ปลายทาง จะไม่สามารถเข้าถึงได้

โดยตรงได้ทันที แต่ ต้องมีการตอบรับจาก Method ใน Object ปลายทางนั้นให้อนุญาตก่อน

Encapsulation

▶ คือแนวความคิดเกี่ยวกับ Class หรือ Object การแยก code ของส่วน

Interface กับ code ส่วนประมวลผล ได้แก่ logic หรือ process และ Data หรือ field ออกจากกัน ดังนั้น Data หรือ Process ทั้งหมด จะถูกซ่อนไว้ โดยกำหนด Interface ที่ดี

-หากมีการเปลี่ยน Code การประมวลผลไปทั้งหมดหรือบางส่วนโปรแกรมก็ยัง
สามารถเรียกใช้งาน Object ได้ตามปกติ

องค์ประกอบของ CLASS

▶ **Class** คือ **Template** ของ **Object**

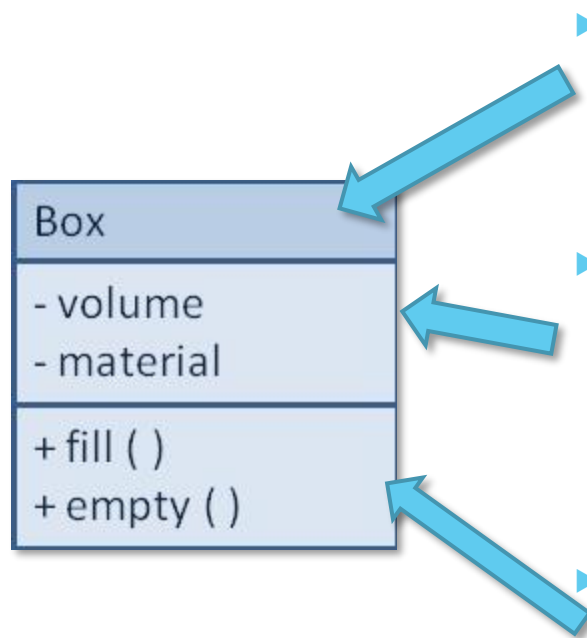
▶ **Class** มีองค์ประกอบได้แก่

a). **ชื่อ Class** : เป็นคำนาม ขึ้นต้นด้วยตัวอักษรตัวใหญ่ ไม่มีช่องว่าง
ตัวอักษรตัวหนา

b). **Attribute**

c). **Method/Operation**

Classes, Attributes, & Operations



▶ Classes

- Templates for instances of people, places, or things

▶ Attributes

- Properties that describe the state of an instance of a class (an object)

▶ Operations

- Actions or functions that a class can perform

องค์ประกอบของ CLASS (ต่อ)

b) Attribute ประกอบด้วย

b.1) Visibility (การเข้าถึงข้อมูล) เป็นเครื่องหมาย ใช้แสดงสิทธิ์การเข้าถึง Attribute ของ Class ได้แก่ Public(+), Private(-), Protect(#)

b.2) ชื่อ Attribute

b.3) เครื่องหมาย :

b.4) ประเภทของ Attribute ได้แก่ Primitive type และ Class Type

c) Method/Operation ประกอบด้วย

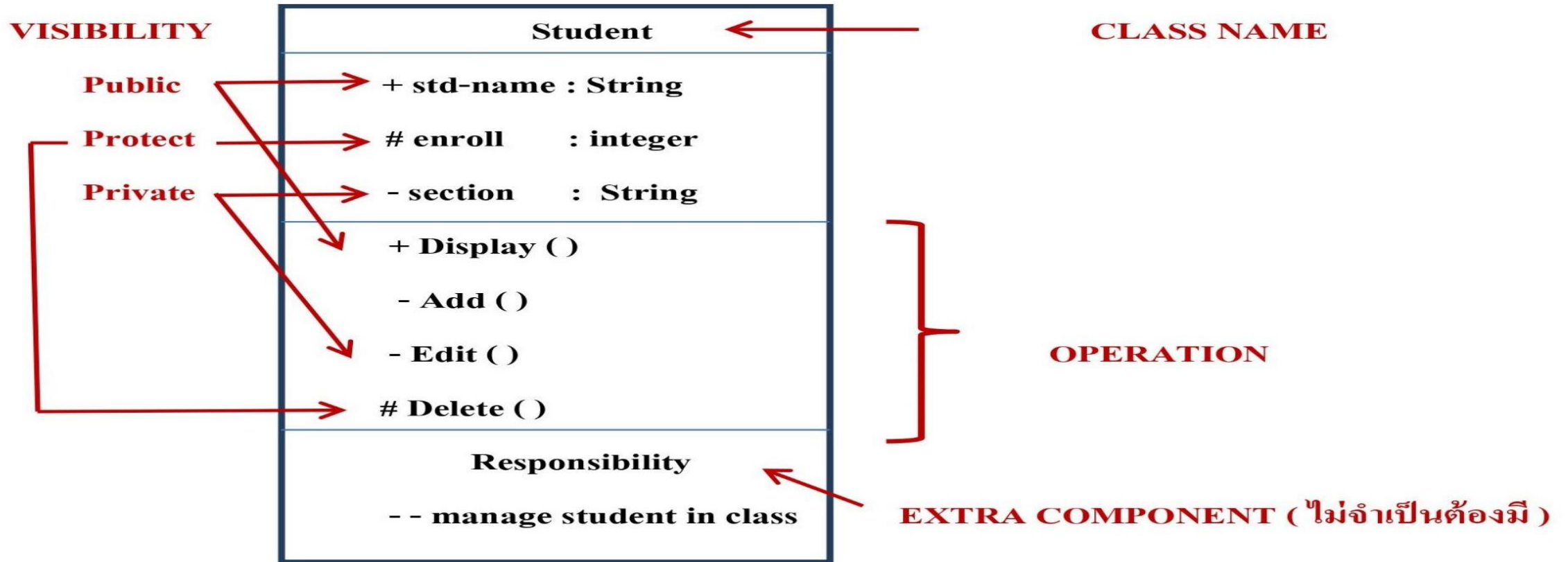
c.1) Visibility เป็นเครื่องหมาย ใช้แสดงสิทธิ์การเข้าถึง Operation ได้แก่ Public(+), Private(-), Protect(#)

c.2) ชื่อของ Method/Operation

c.3) Parameter อยู่ภายในเครื่องหมายวงเล็บ เป็นตัวแปร หรือ Object ที่ถูกส่งเข้าไปใน Method

c.4) Return Type อยู่ต่อจากเครื่องหมาย : เป็นชนิดของผลลัพธ์ที่ได้จากการดำเนินงาน ซึ่งจะถูกส่งออกมาสู่ภายนอก

- ▶ **+** หรือ ระดับ **Public** หมายถึง Attribute นี้ จะยอมให้คลาสอื่นเข้าถึงและเรียกใช้งานได้ตลอด
- ▶ **-** หรือ ระดับ **Private** หมายถึง Attribute นี้ ไม่สามารถมองเห็นได้จากภายนอกของ Class แต่สามารถมองเห็นได้ภายในของ Class เองเท่านั้น
- ▶ **#** หรือ ระดับ **Protected** หมายถึง Attribute ที่สงวนไว้สำหรับการทำ Inheritance โดยเฉพาะ Attribute เหล่านี้จะเป็นของ Superclass เมื่อ Inherit เสร็จแล้ว จะกลายเป็น private attribute ของ Subclass ต่อไป)



Visibility ของ attribute หรือ Visibility ของ Method ได้แก่

Public (+) สามารถเข้าถึงได้โดยตรงจากภายนอก (Public attributes are visible to all classes)

Private(-) ถูกใช้งานจากภายใน Class เท่านั้น (Private attributes are visible only to an instance of the class in which they are defined) (ต่อ)

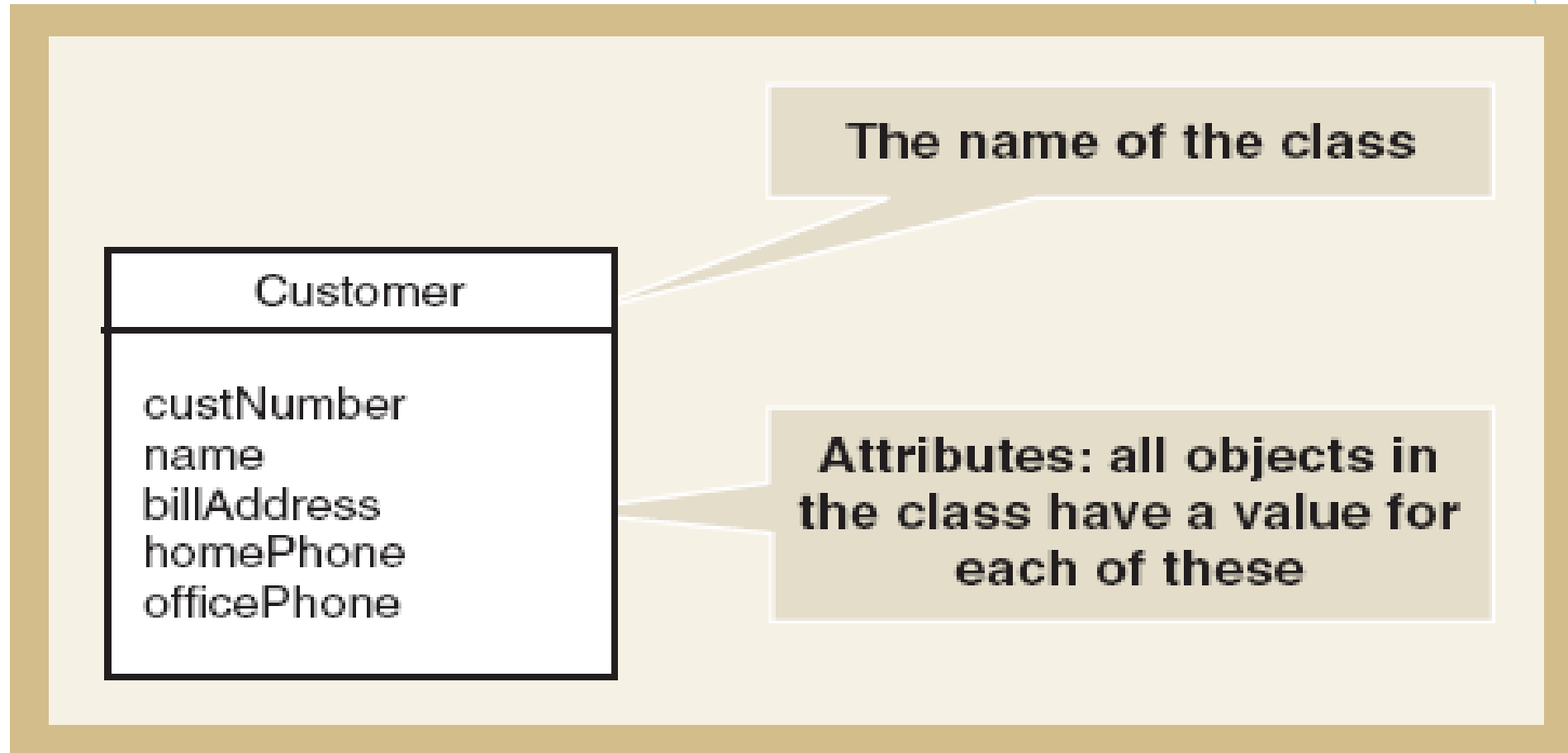
Protected(#) สามารถเห็นหรือเข้าถึงได้จากภายในSubclass เท่านั้น

(Protected attributes are like private attributes, but are also visible to descendant classes

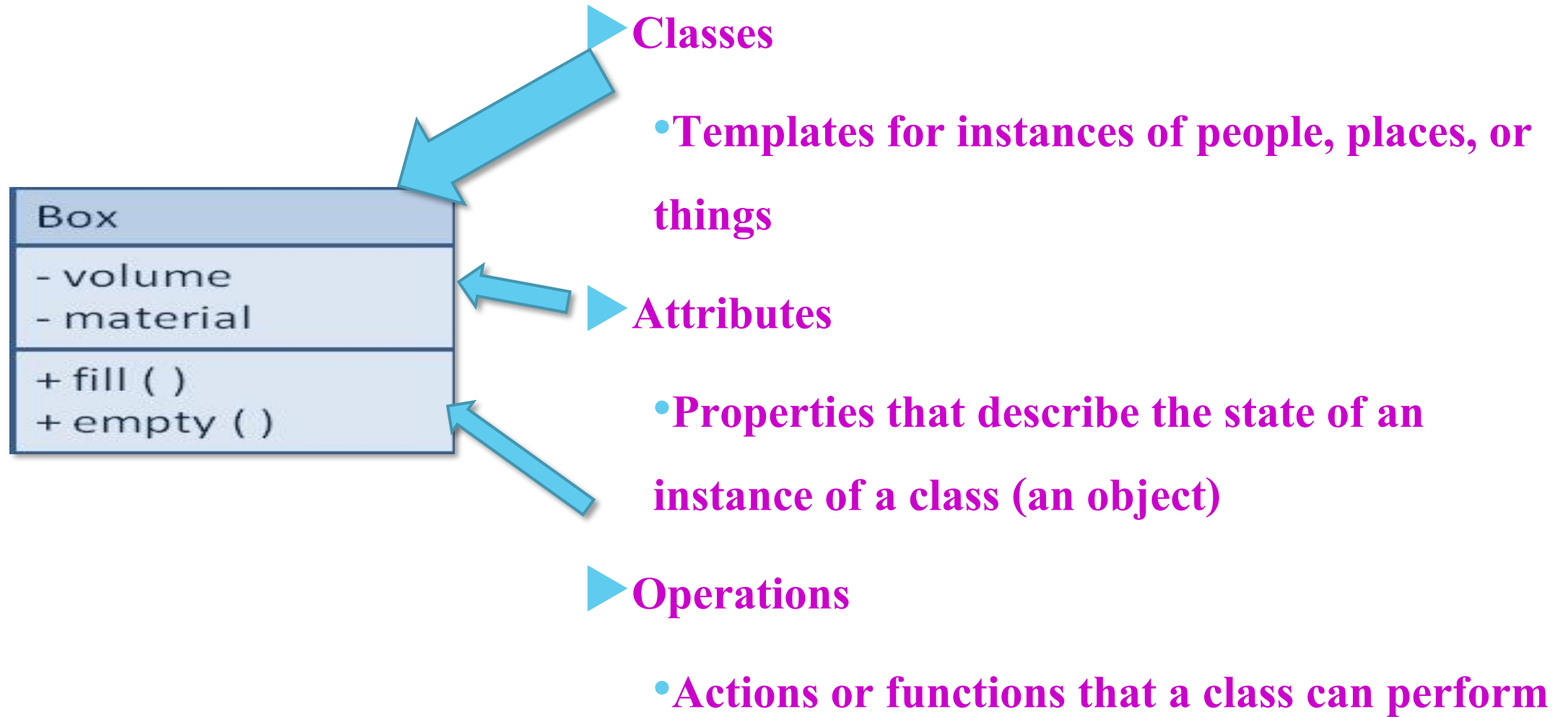
ตัวอย่าง Object “ Customer ”

Attribute	Method	State	Identity	Responsibility
Email	Create Order	-Not order	Email	-Enter order
Name	Search order	-Ordered		- Paid
Surname	Search available-item	-Paid		
Card-id	Pay	-not paid		
Mobile-number	Receive item			
address	search payment status			

UML Class Symbol



Classes, Attributes, & Operations



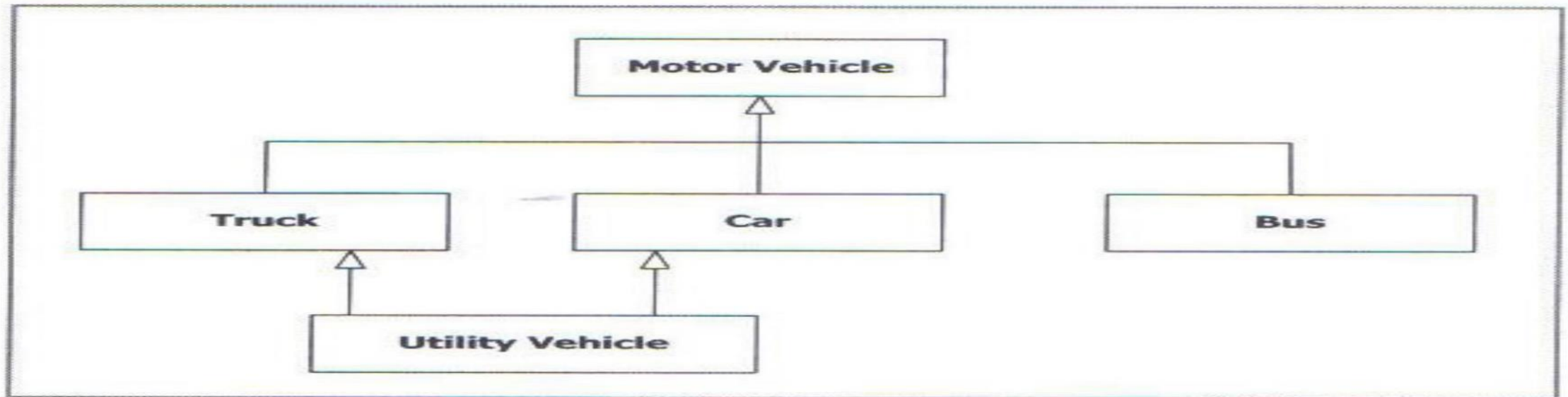
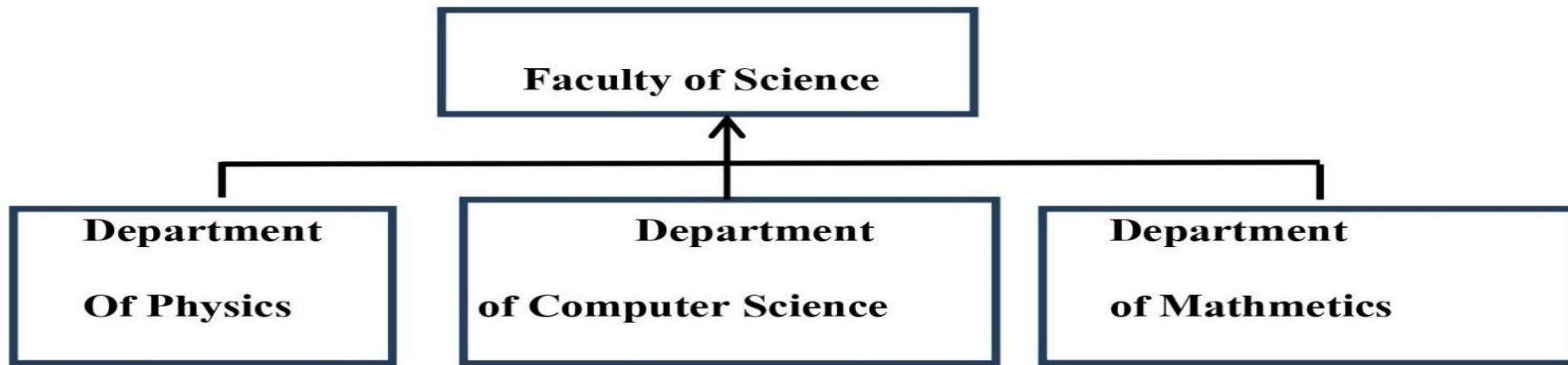
- การสืบทอดคุณสมบัติ (INHERITANCE) = การกำหนดคุณสมบัติของออบเจกต์แต่ละตัว

ในระบบ จะใช้วิธีการสืบทอด (Inheritance) โดยอาศัยคุณสมบัติของออบเจกต์ที่มีอยู่แล้วใส่ลง
ในออบเจกต์ตัวใหม่ แนวความคิดเชิงวัตถุจะถือว่าการสืบทอดเป็นสิ่งสำคัญ เพราะว่ามีสิ่งใด
ในโลกที่เกิดขึ้นเอง ต้องมีการสืบทอด เพื่อ

ทำให้มีโครงสร้างเป็นระบบ ระเบียบ ปรับเปลี่ยนได้ง่าย

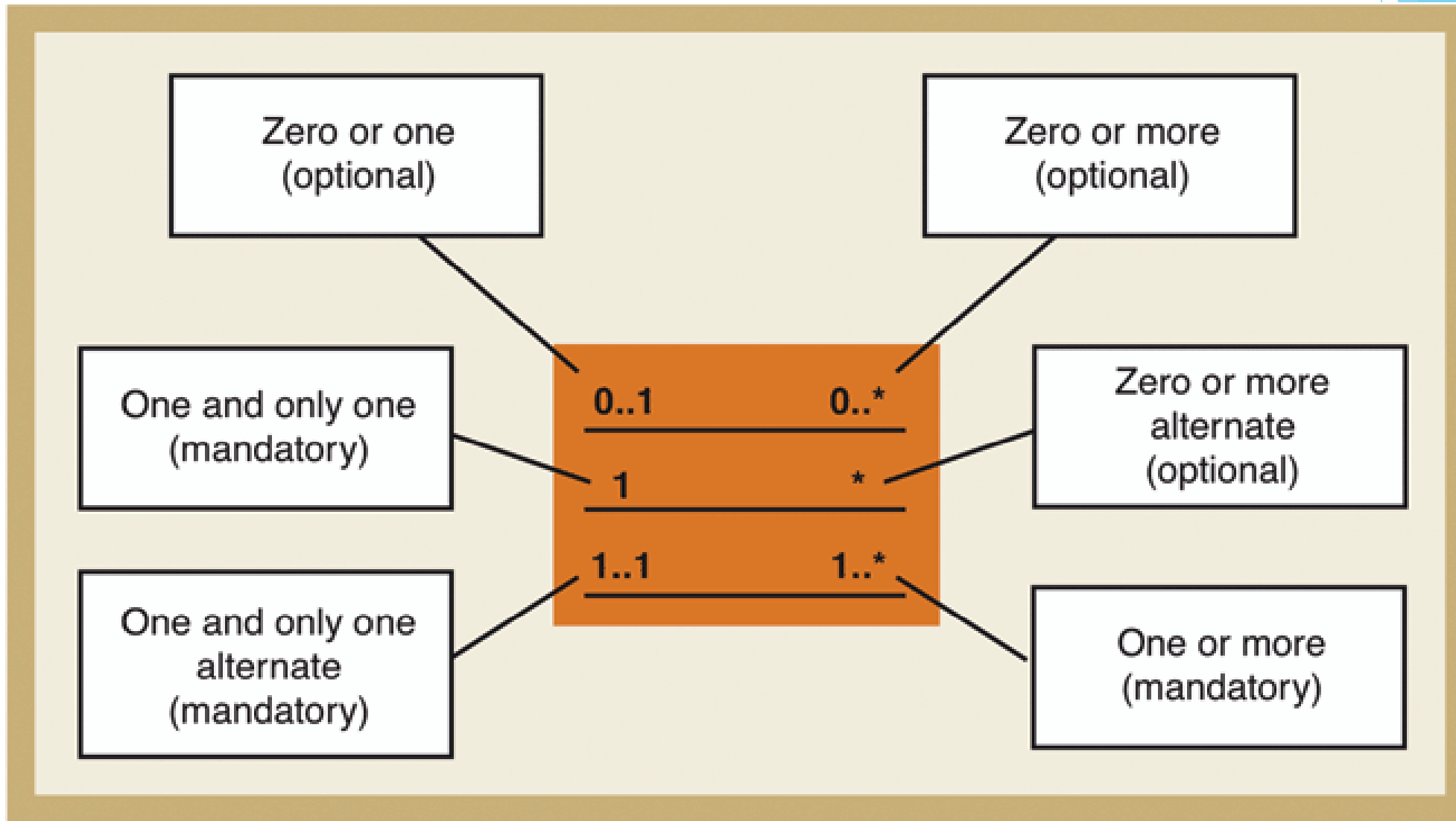
ลดเวลาในการพัฒนาระบบ

ลดค่าใช้จ่ายในการพัฒนา

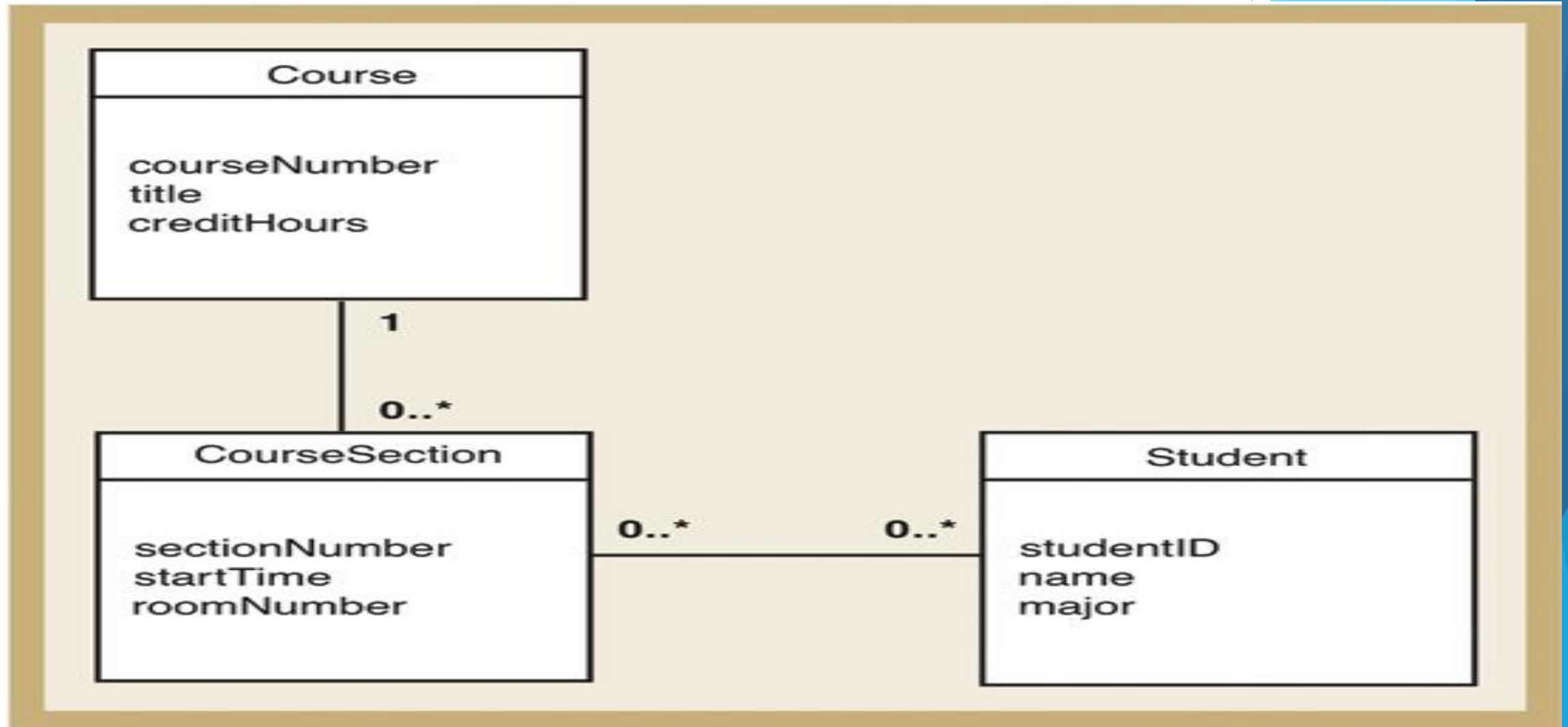


ตัวอย่าง : การสืบทอดคุณสมบัติของ Subclass ยังสามารถสืบทอดจาก Superclass มากกว่าหนึ่งก็เป็นได้ ที่เรียกว่า **Multiple Inheritance**

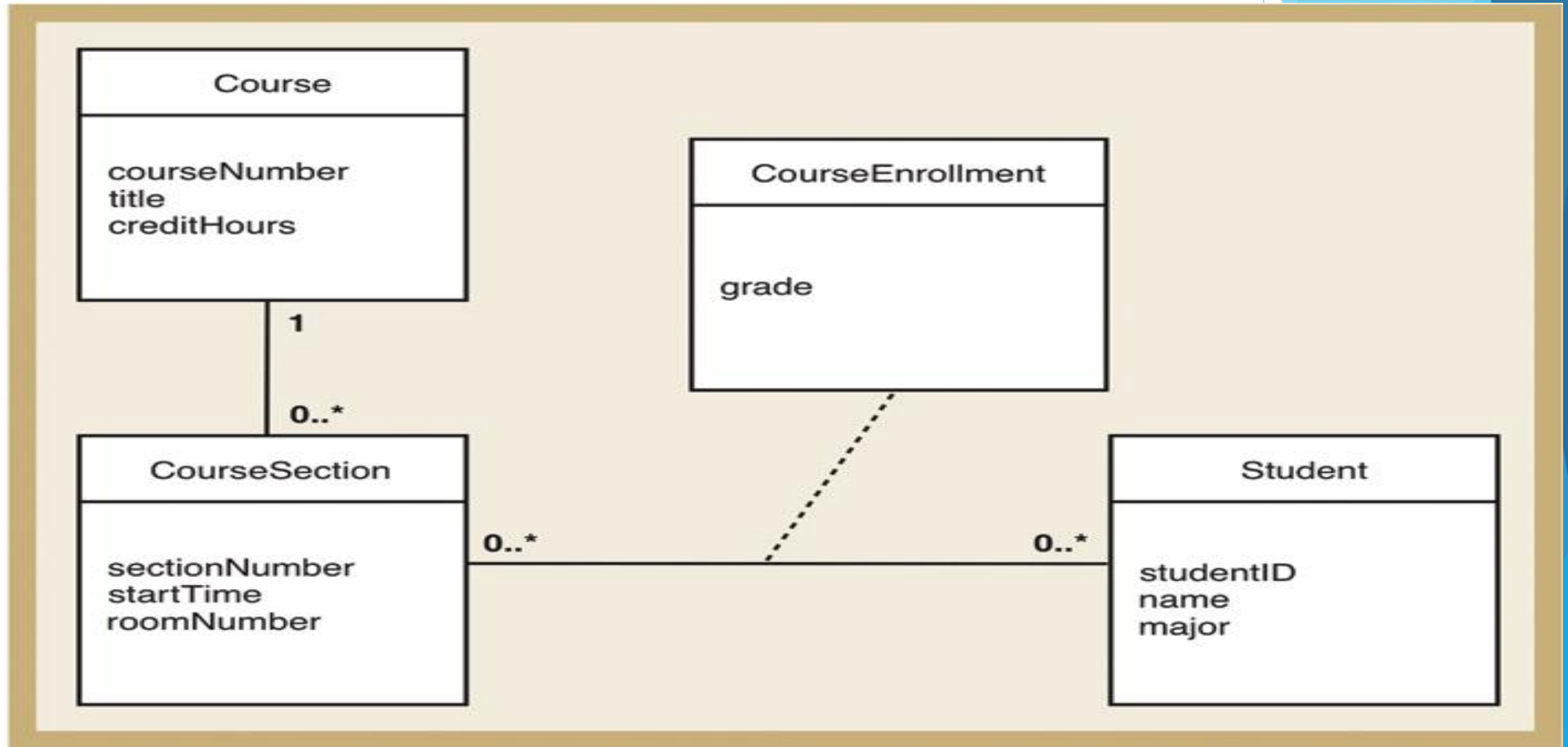
Multiplicity of Associations



University Course Enrollment Domain Model Class Diagram



Association Class and Grade Attribute



5) **FUNCTION** = ความสามารถในการทำกิจกรรม ของ OBJECT ที่มีไว้เพื่อให้ object อื่นใน DOMAIN สามารถเรียกใช้

6) **ABSTRACT** = กระบวนการในการให้ CONCEPT และ OBJECT จนเกิดเป็น CLASS เพื่อ

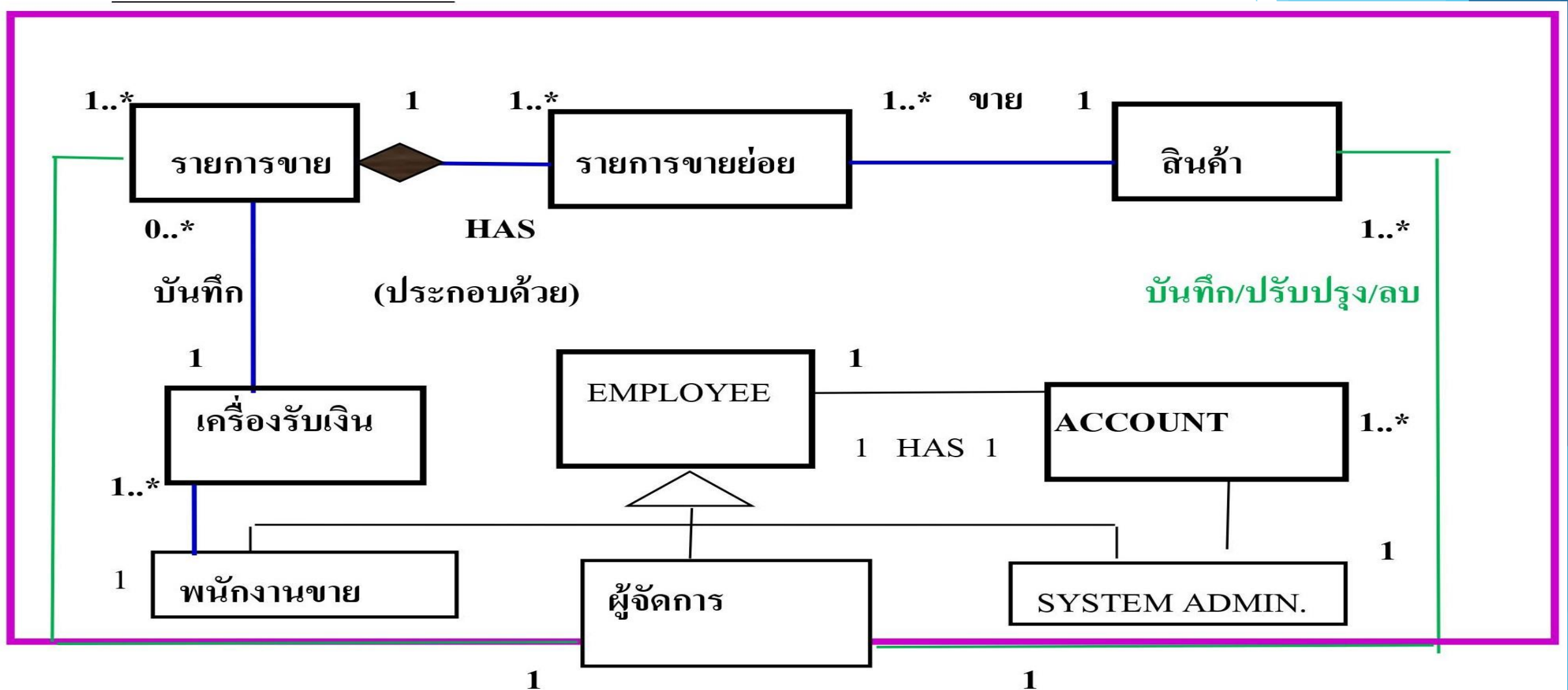
- แยกประเภท Object ต่างๆ ที่อยู่ใน Domain
- ให้ Concept กับ Object ต่างๆ เหล่านั้น เพื่อให้ได้ Class พื้นฐาน

6.1) **AGGREGATION** = นำเอา Class พื้นฐานมารวมกัน มาเป็น Class ใหญ่ขึ้น

= ความสัมพันธ์แบบ “Whole-Part” หรือ “is part of”

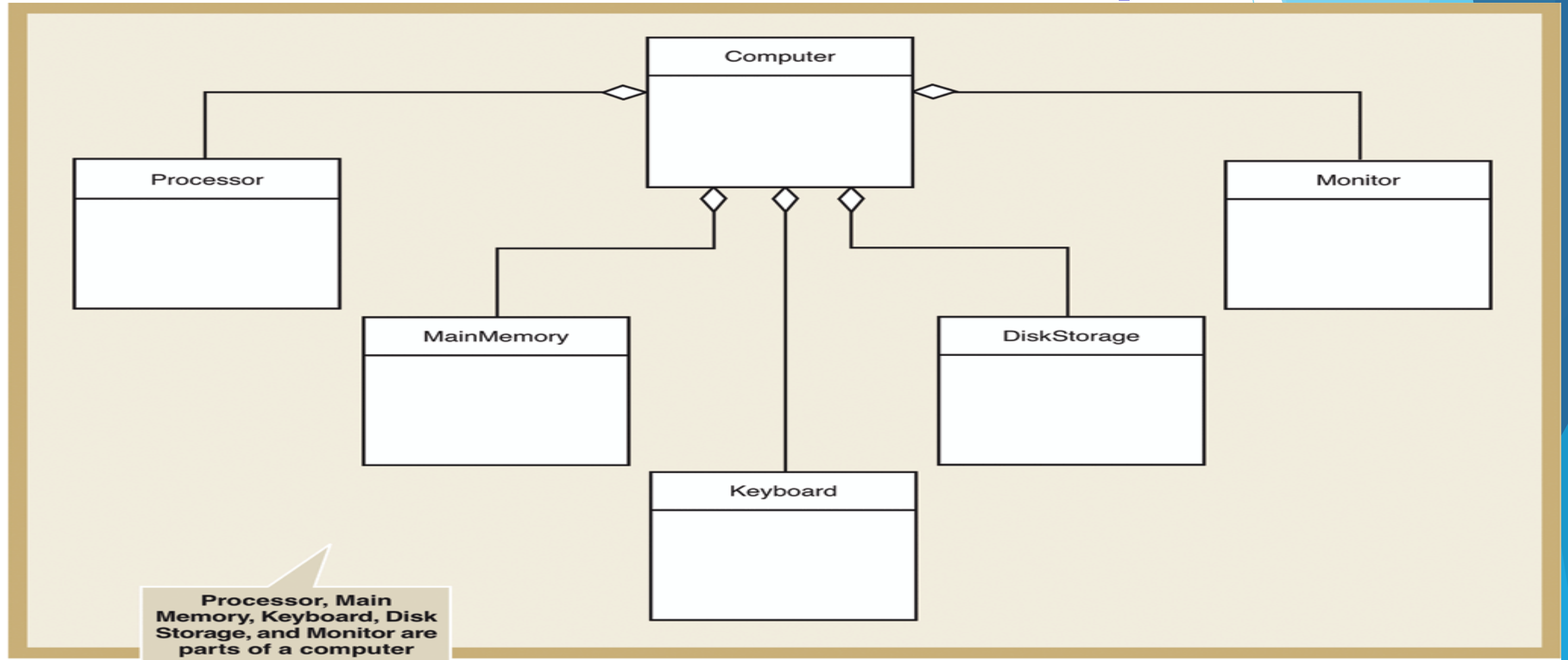
มีคลาสใหญ่ที่สุด เป็นหลัก และมีคลาสอื่น เป็นส่วนหนึ่งของ Class หลัก

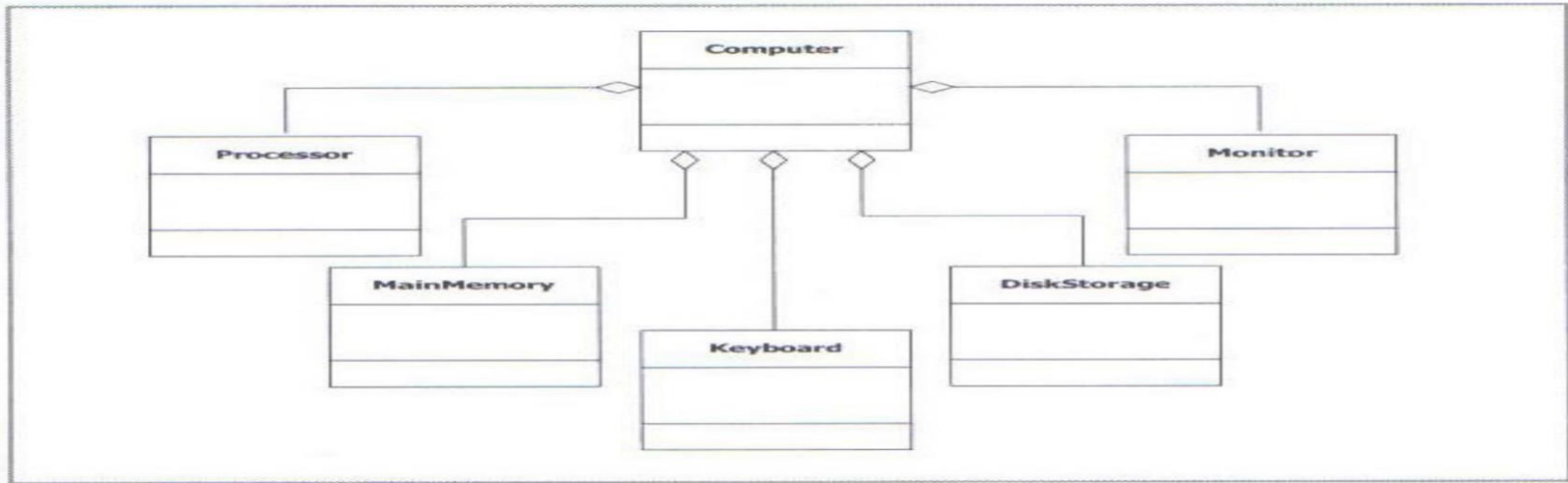
CLASS DIAGRAM : Conceptual Model



Whole-Part Aggregation Relationships :

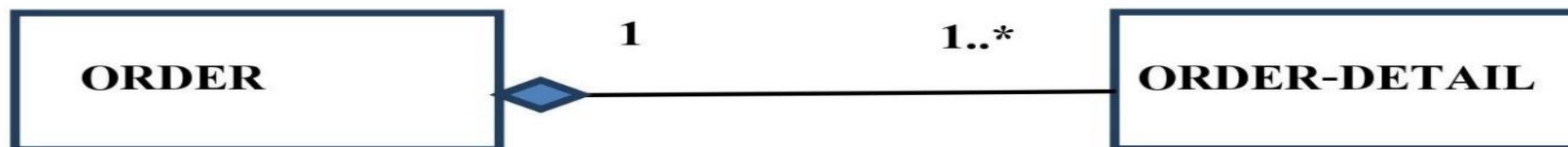
เมื่อลบ **Whole class** แล้ว **Part class** จะไม่ถูกลบไปด้วย



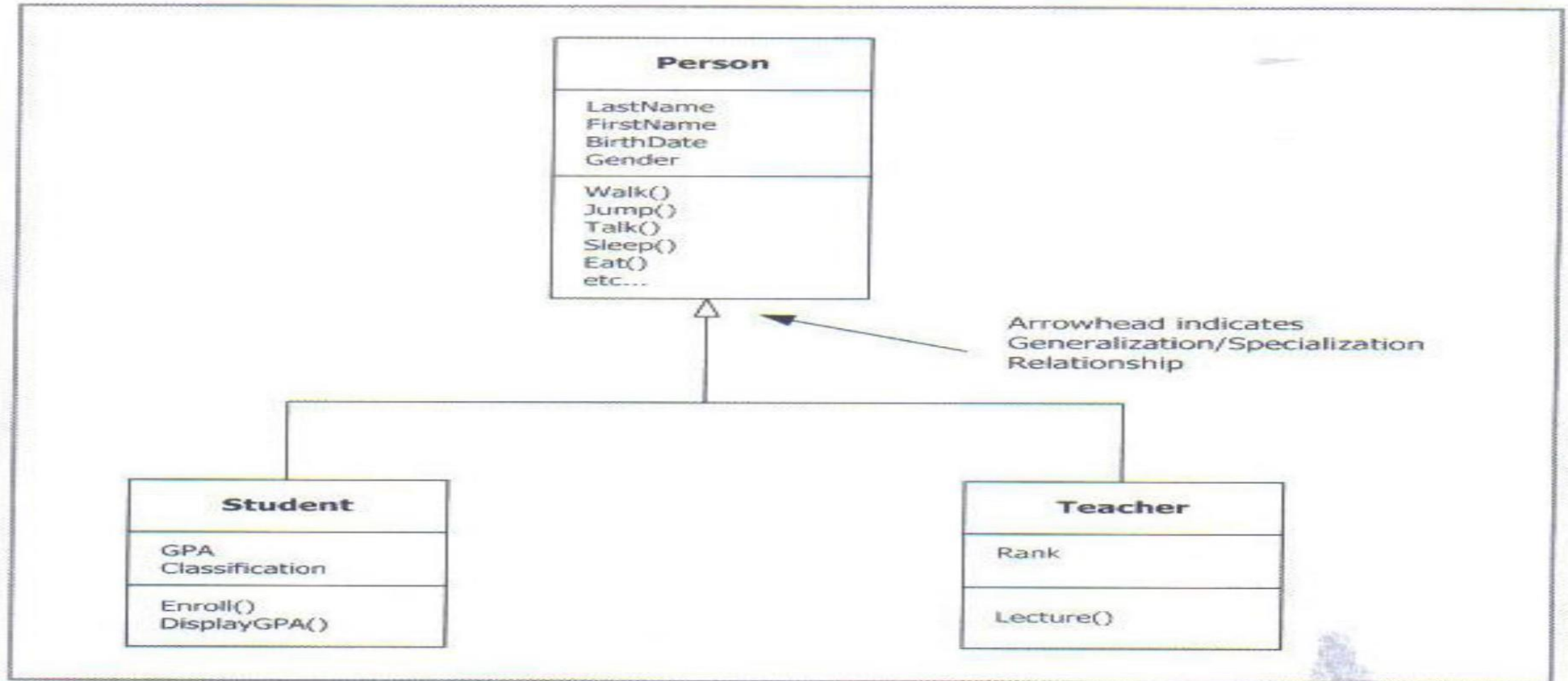


6.2) **Composition** เป็นความสัมพันธ์แบบ**ขึ้นต่อกัน** และมีความเกี่ยวข้องกันเสมอ

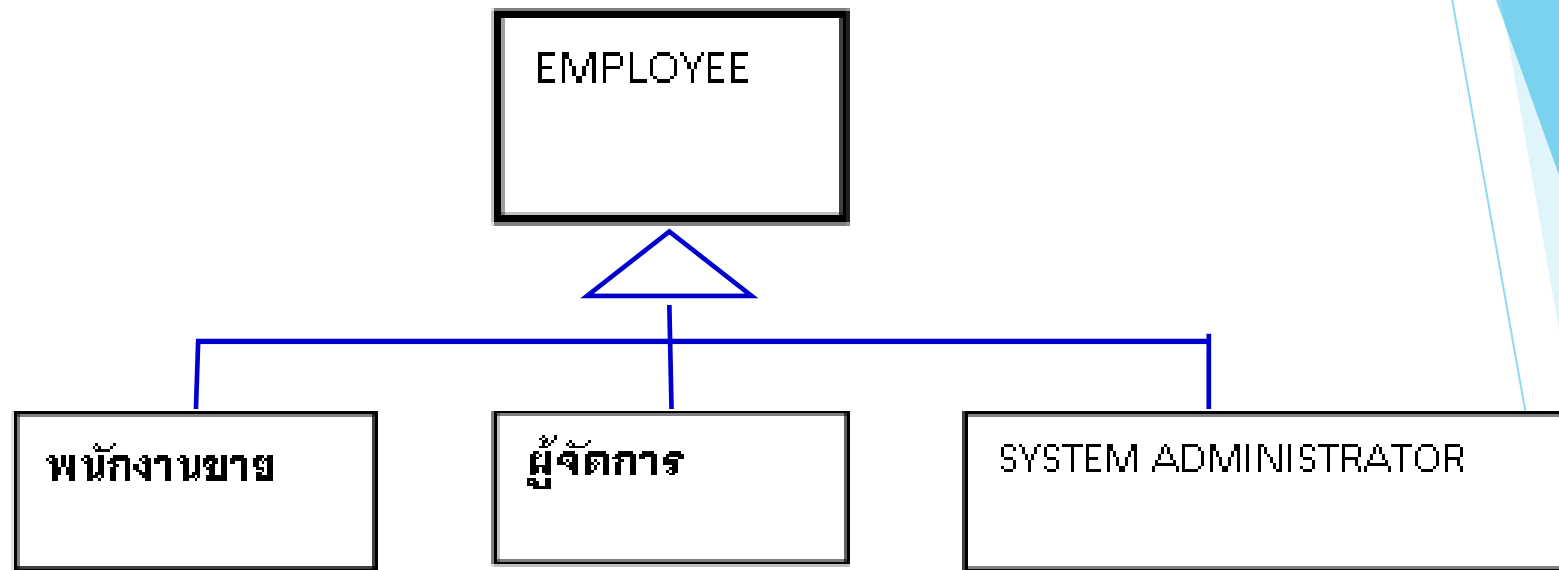
หรือ Subclass เป็นส่วนประกอบของ Superclass ที่ขึ้นอยู่กับกันเสมอ
 หากไม่มี ข้อมูลหรือรายการใน Subclass จะมีผลกระทบทำให้ ไม่มีข้อมูล
 หรือรายการ นี้ใน Superclass ด้วยเช่นกัน



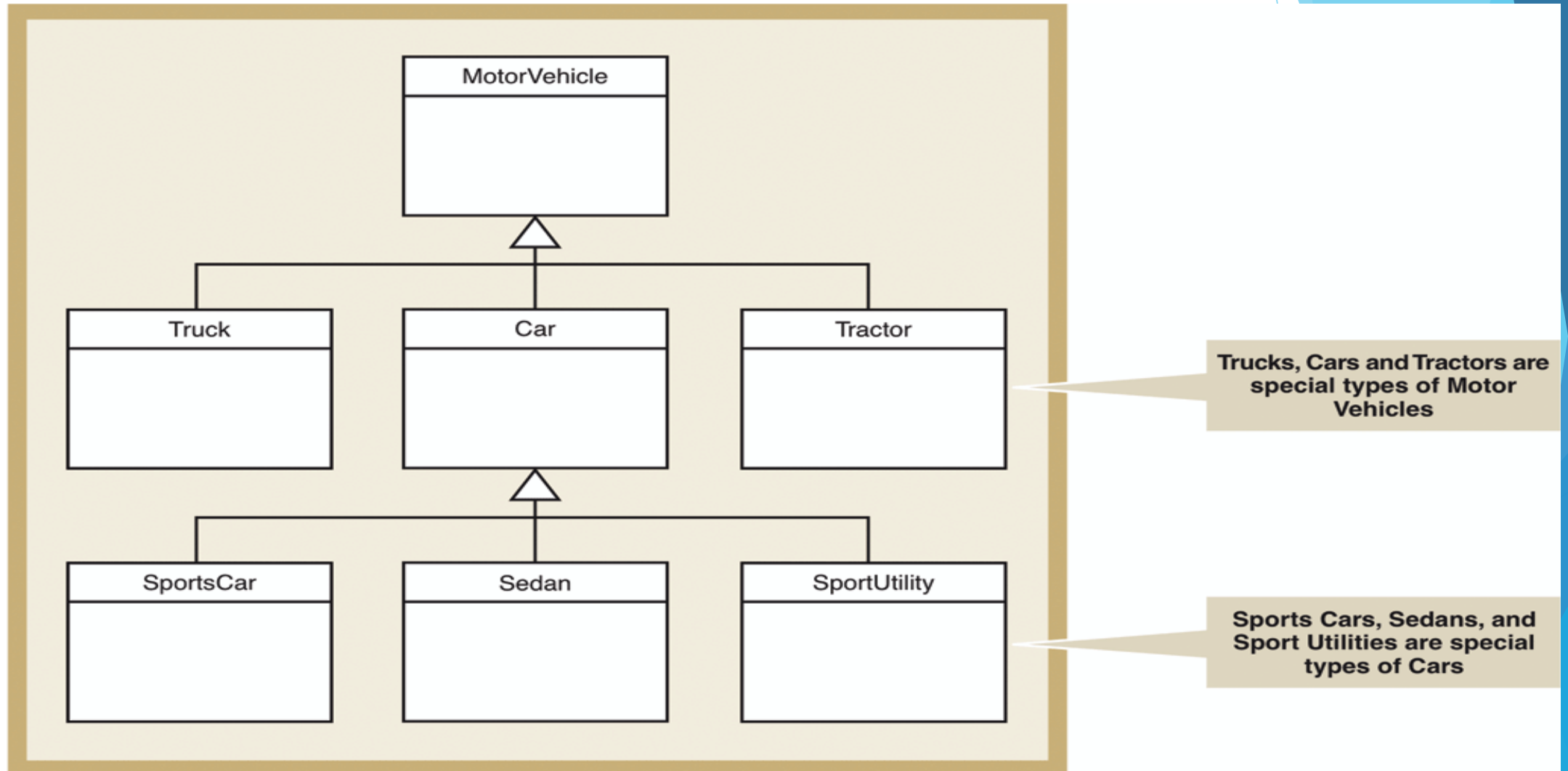
6.3) Generalization = นำ Class ที่มีคุณสมบัติคล้ายกันมา จัดหมวดหมู่ เป็นเทคนิคที่นำ
คุณสมบัติ และพฤติกรรมของ Superclass ถ่ายทอดคุณสมบัติไปยัง
Subclass



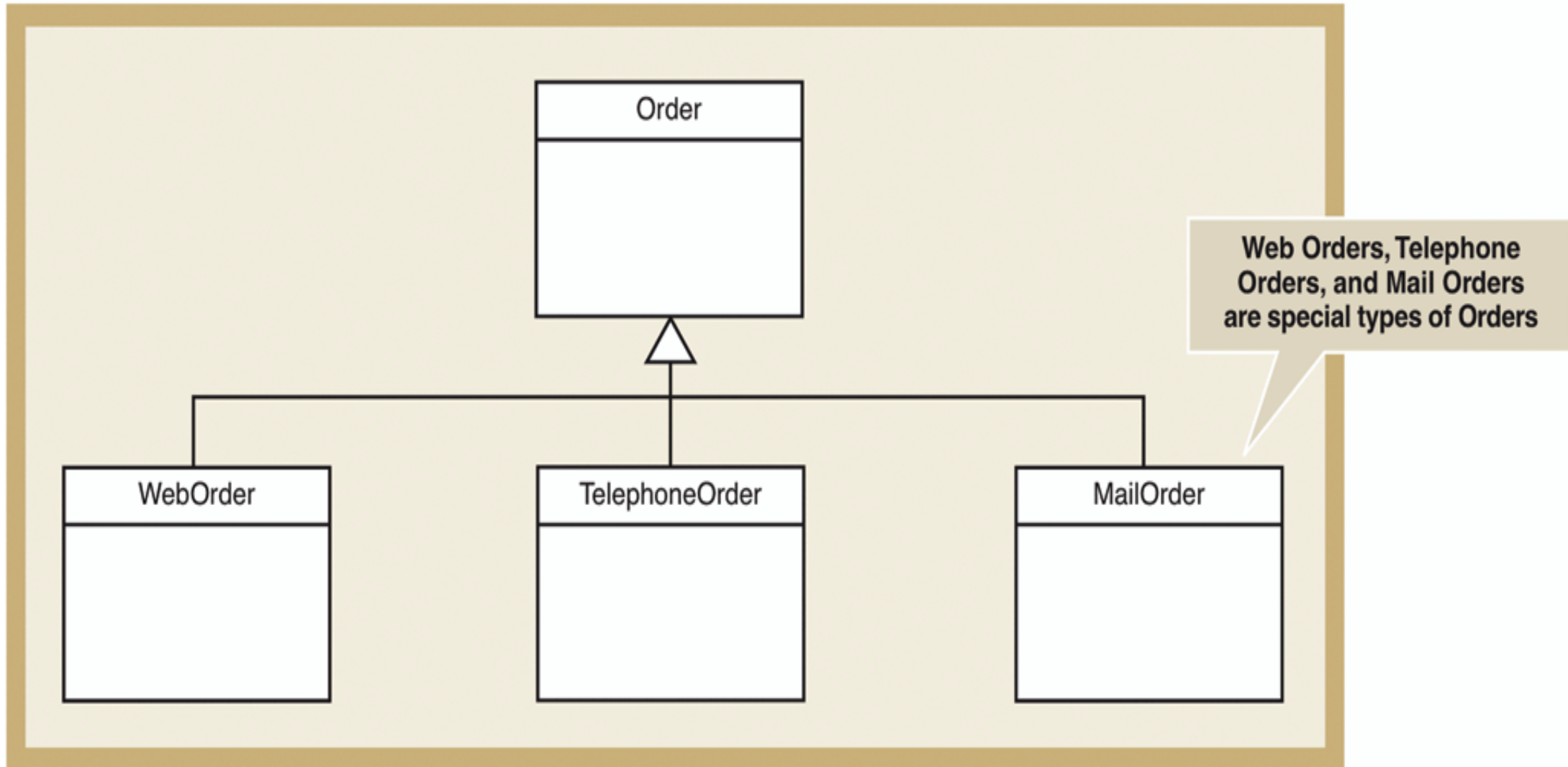
Generalization



A Generalization/Specialization Class Hierarchy for Motor Vehicles



A Generalization/Specialization Class Hierarchy for RMO Orders



Polymorphism

- ▶ เป้าหมายของ **Polymorphism** คือ ช่วยให้เราสามารถนำ CLASS เดิม ที่มีอยู่ แล้ว มา Reuse และใช้ประโยชน์ได้ แต่ถ้า Class นั้นมีคุณสมบัติ ที่ไม่ตรงตาม System Requirements แล้ว Developer ก็สามารถเปลี่ยนแปลงหรือเพิ่ม ความสามารถของ Derived class โดยไม่ต้องไปเปลี่ยน Base Class
- ▶ สิ่งที่เปลี่ยนแปลงรูปร่างได้ก็คือ **Method** ที่ Inherit มาจาก Base class (คลาสแม่) โดยการเปลี่ยนแปลง บาง Method ของ Base Class ให้ตรงกับ ความต้องการของระบบ เราเรียกว่าวิธีนี้ว่า Method Override หรือสร้าง Method ใหม่ ทับ Method เดิม เราเรียกว่าวิธีนี้ว่า Method Hiding

7 Steps to Structural Models

1. **Create CRC (CRC Cards)**
2. **Examine common object lists**
3. **Role-play the CRC cards**
4. **Create the class diagram**
5. **Review the class diagram**
6. **Incorporate patterns**
7. **Review the model**

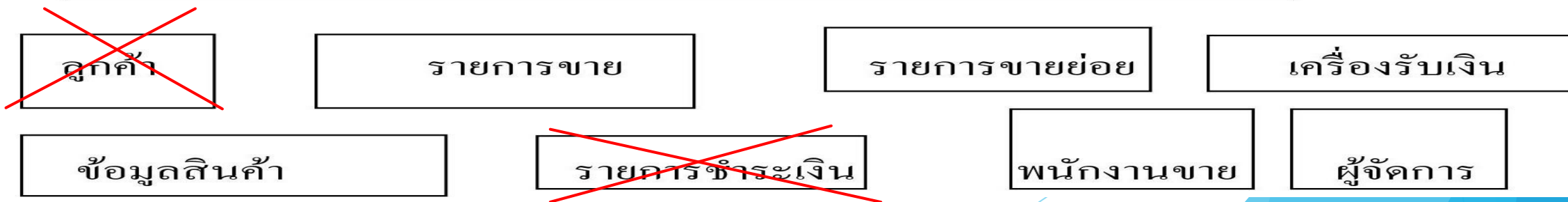
CRC (Class Responsibilities and Collaborators)

- ▶ แต่ละระบบงาน มีหลายคลาส แต่ละคลาสต่างมีความสัมพันธ์กัน และมี การประสานงานและทำงานร่วมกัน(collaboration) ตาม System Requirements และเป้าหมายของการพัฒนาระบบ
- ▶ ดังนั้น แต่ละ class ซึ่งมี Attributes และ Operations แล้ว ยังต้องมี Relationships และ Collaborations แสดงว่าคลาสนั้นมีความสัมพันธ์กับ class อื่น และทำงานร่วมกับ class อื่น อย่างไร

CLASS DIAGRAM

ค้นหา CLASS

PROCESS	รายการขาย <u>รายการชำระเงิน</u> รายการขายย่อย เครื่องรับเงิน ข้อมูลสินค้า
ACTOR	ลูกค้า พนักงานขาย ผู้จัดการ
REPORT/OUTPUT	ใบเสร็จรับเงิน รายงานสรุปการขาย
EVENT	การขาย การชำระเงิน
LOCATION	จุดเครื่องรับเงิน



CRC

CLASS : Saleclerk

Responsibilities

1. Create Sales
 2. Create Sale-details
 3. Produce Summary
 4. Search Sales,
 - 5 Search Sale-details
 6. Search item
-

Collaborator

- 1) Sales
- 2) Sale-details
- 3) Cash

Attribute

Card-id
Name
Surname
Salary
Email
.....

Relationships

Generalization:
Employee

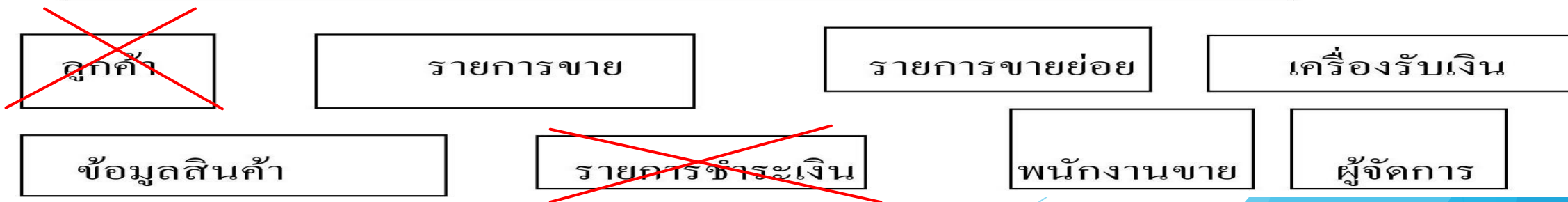
Aggregation :

Association: Sales
Sale-details
Cash

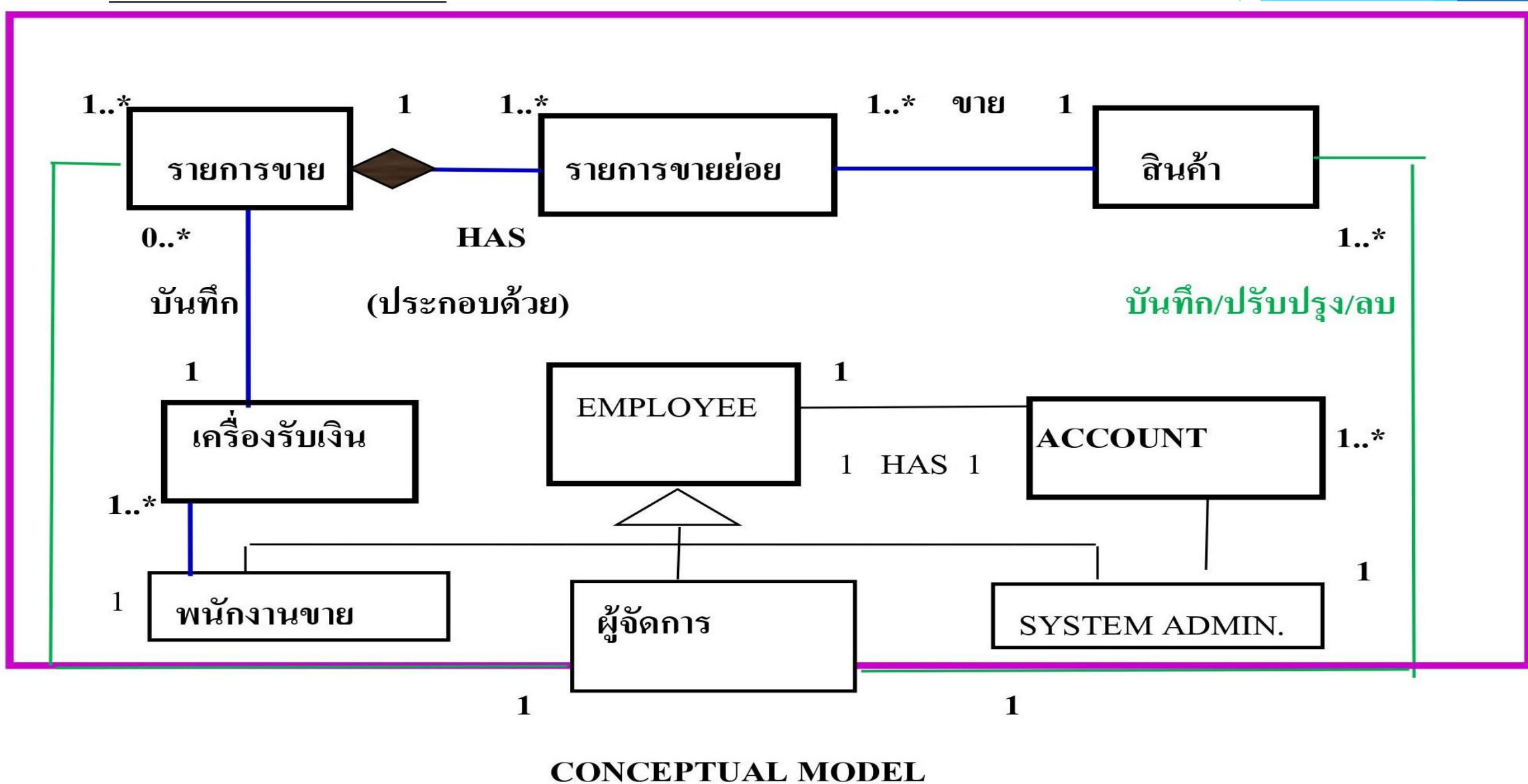
CLASS DIAGRAM

ค้นหา CLASS

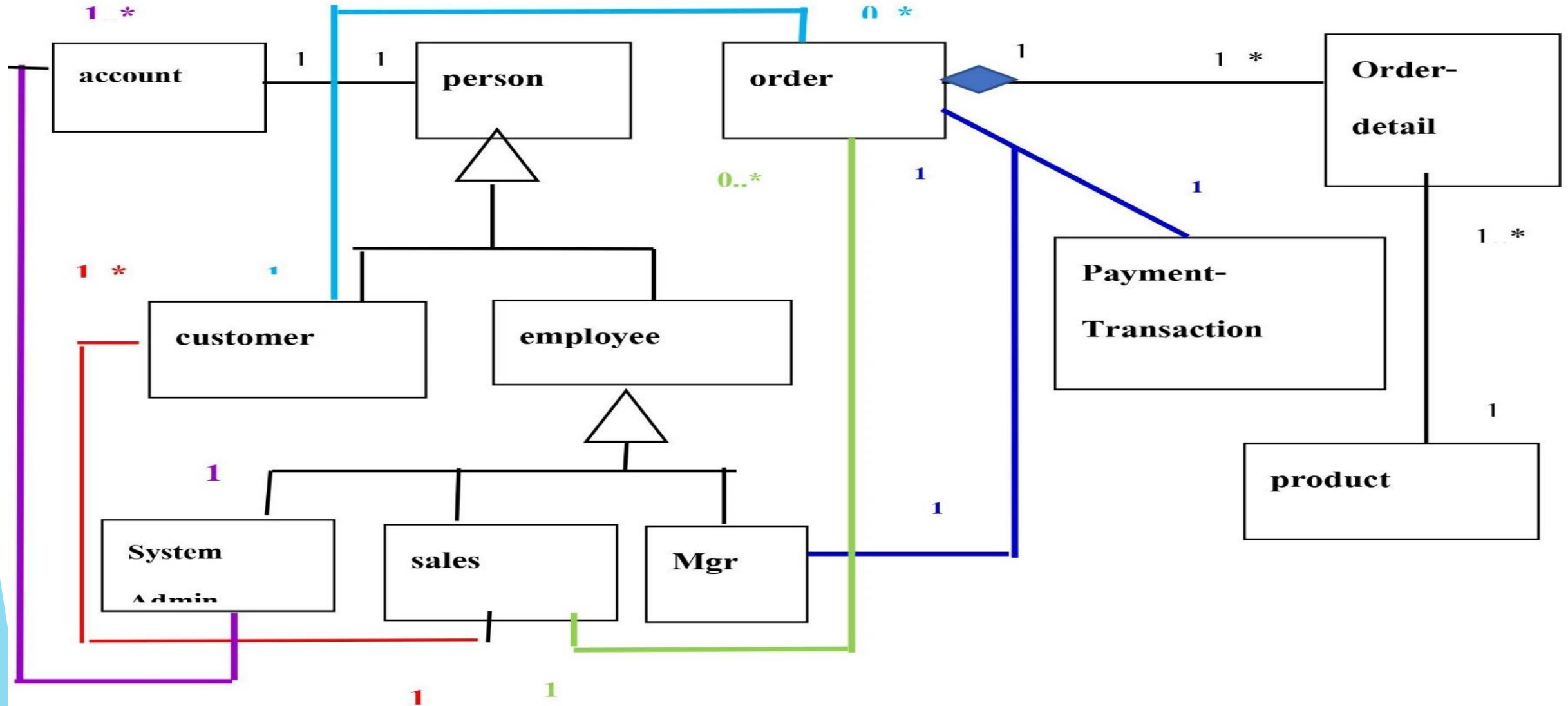
PROCESS	รายการขาย <u>รายการชำระเงิน</u> รายการขายย่อย เครื่องรับเงิน ข้อมูลสินค้า
ACTOR	ลูกค้า พนักงานขาย ผู้จัดการ
REPORT/OUTPUT	ใบเสร็จรับเงิน รายงานสรุปการขาย
EVENT	การขาย การชำระเงิน
LOCATION	จุดเครื่องรับเงิน



Conceptual Model



Conceptual Model



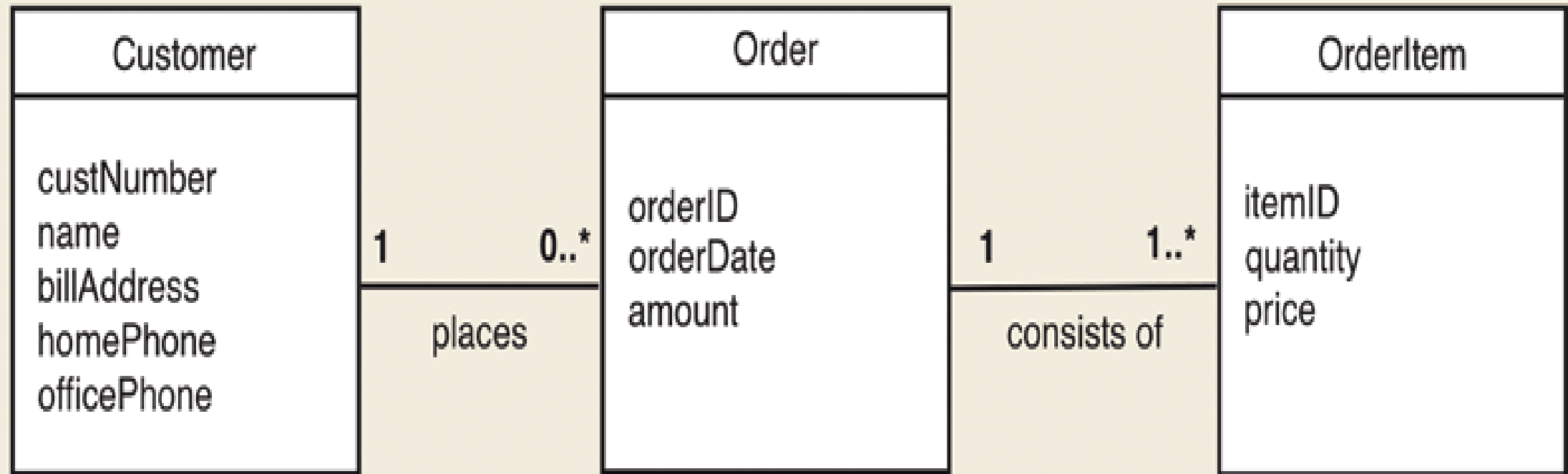
Front:

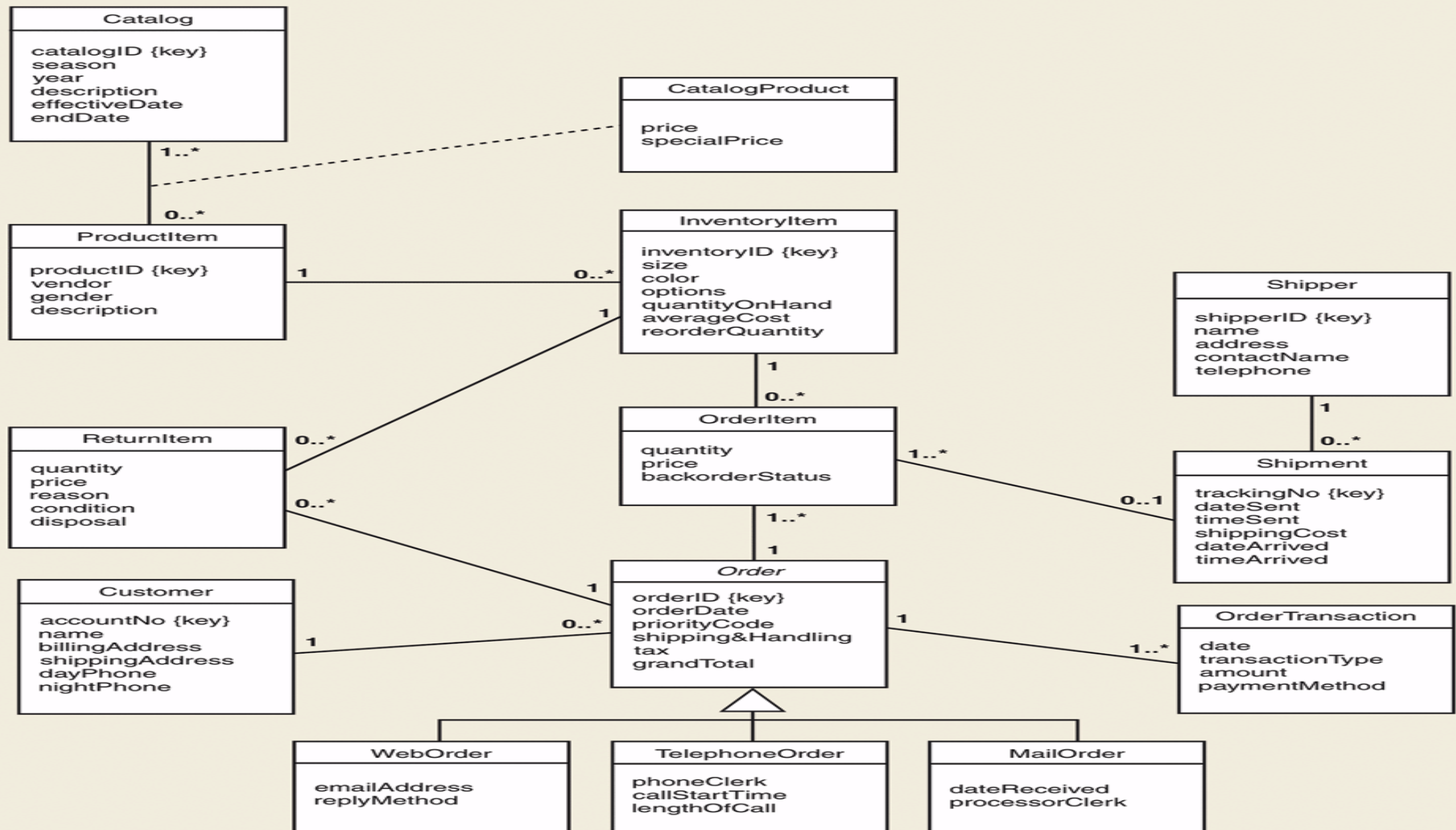
Class Name: Patient	ID: 3	Type: Concrete, Domain
Description: An individual that needs to receive or has received medical attention		Associated Use Cases: 2
Responsibilities Make appointment Calculate last visit Change status Provide medical history 		Collaborators Appointment Medical history

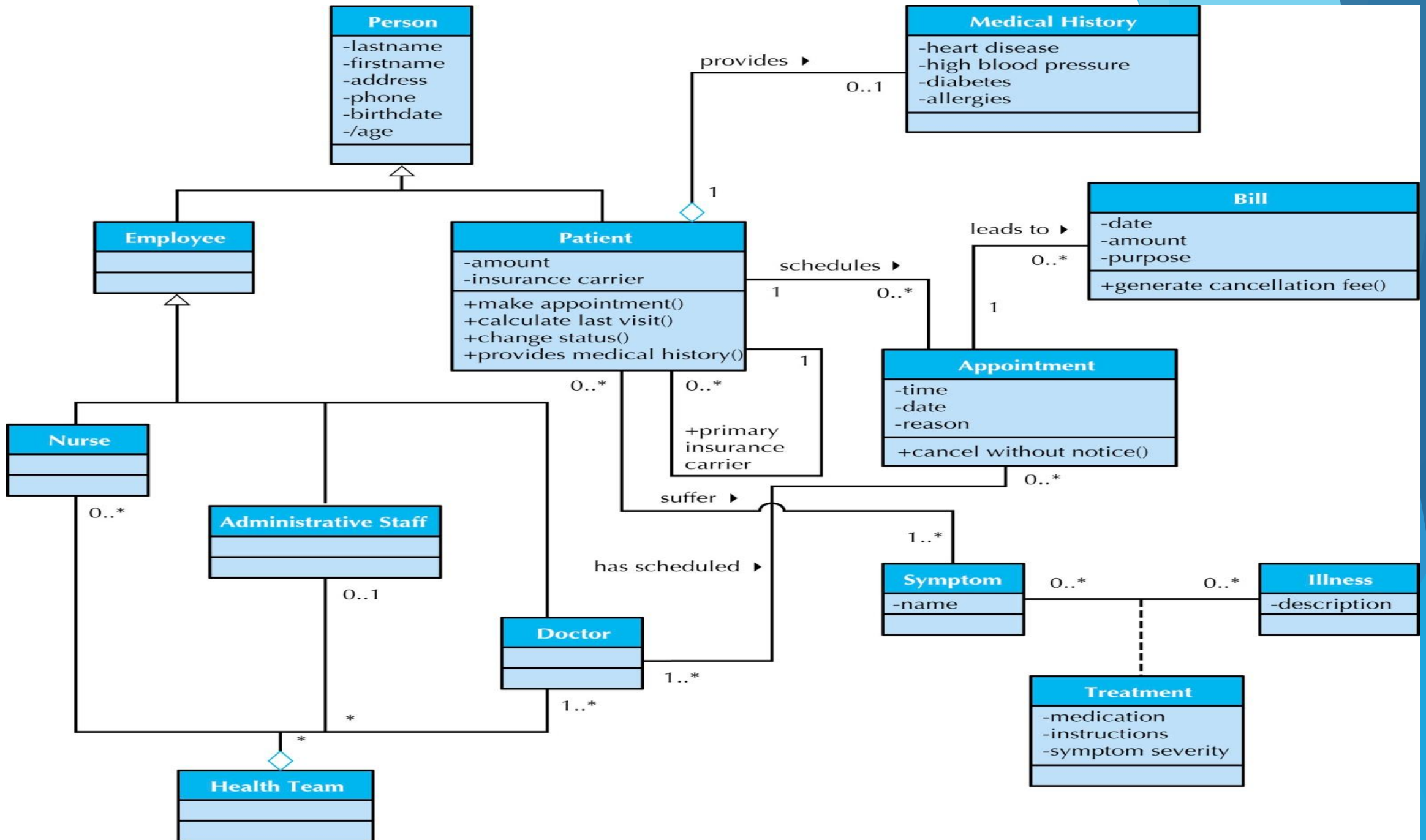
Back:

Attributes: Amount (double) Insurance carrier (text) 	
Relationships: Generalization (a-kind-of): Person Aggregation (has-parts): Medical History Other Associations: Appointment 	

Simple Domain Model

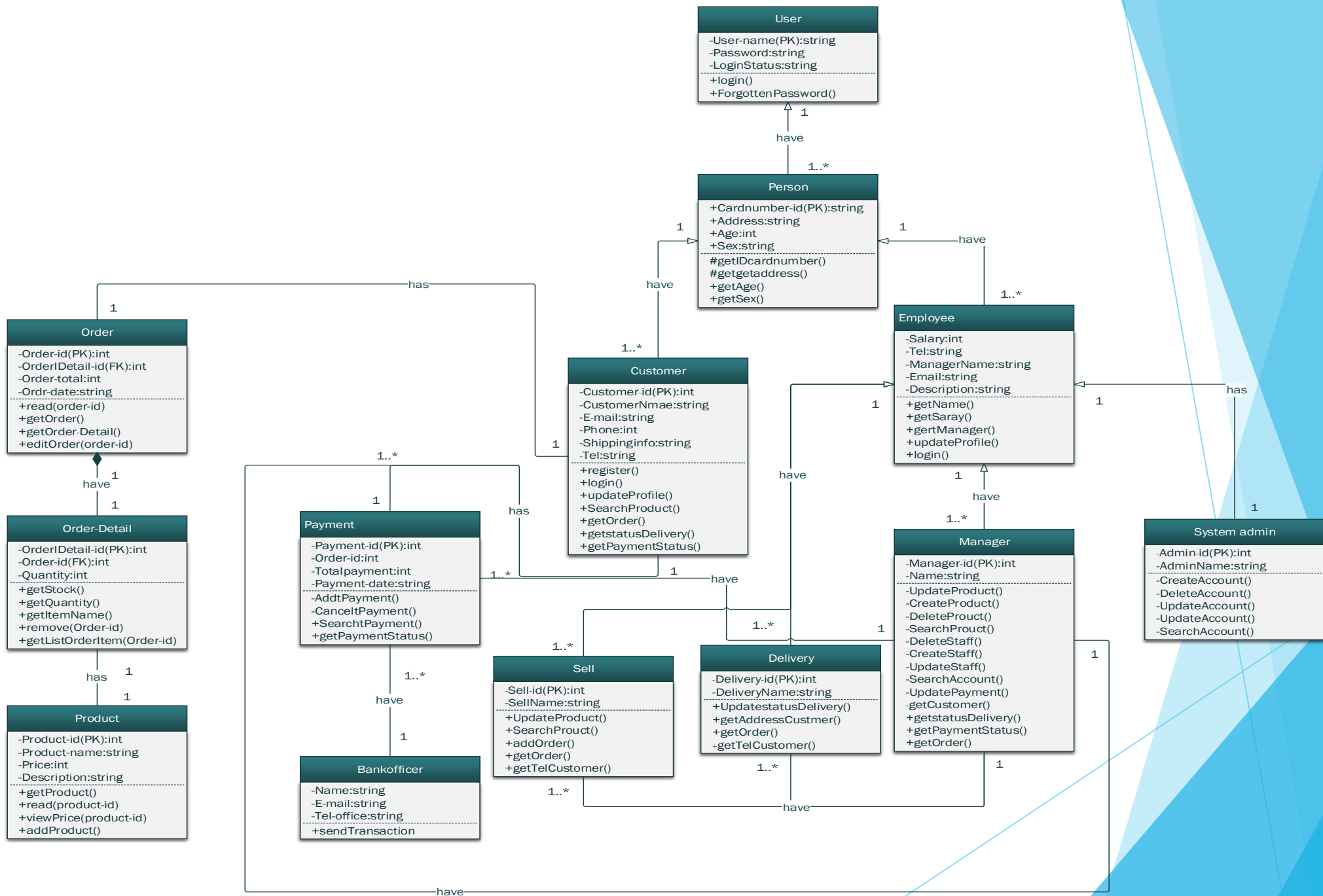






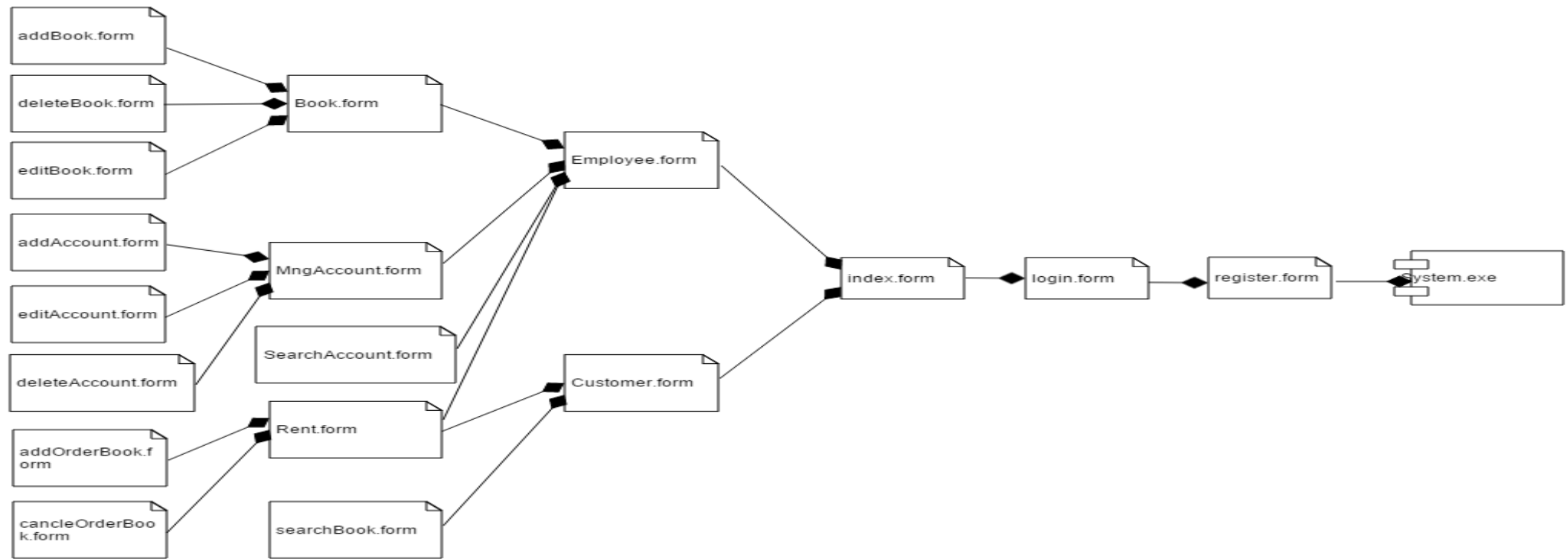
User Info

No	Key	Name	Data Type	Length	Description
1	PK	USER_NAME	char	10	ชื่อของ user สำหรับ login
2		USER_PASSWORD	char	10	Password ของ user สำหรับ login
3		USER_TYPE	char	1	C = customer , E = Employee
4		USER_EMAIL	char	50	E-mail ของ user

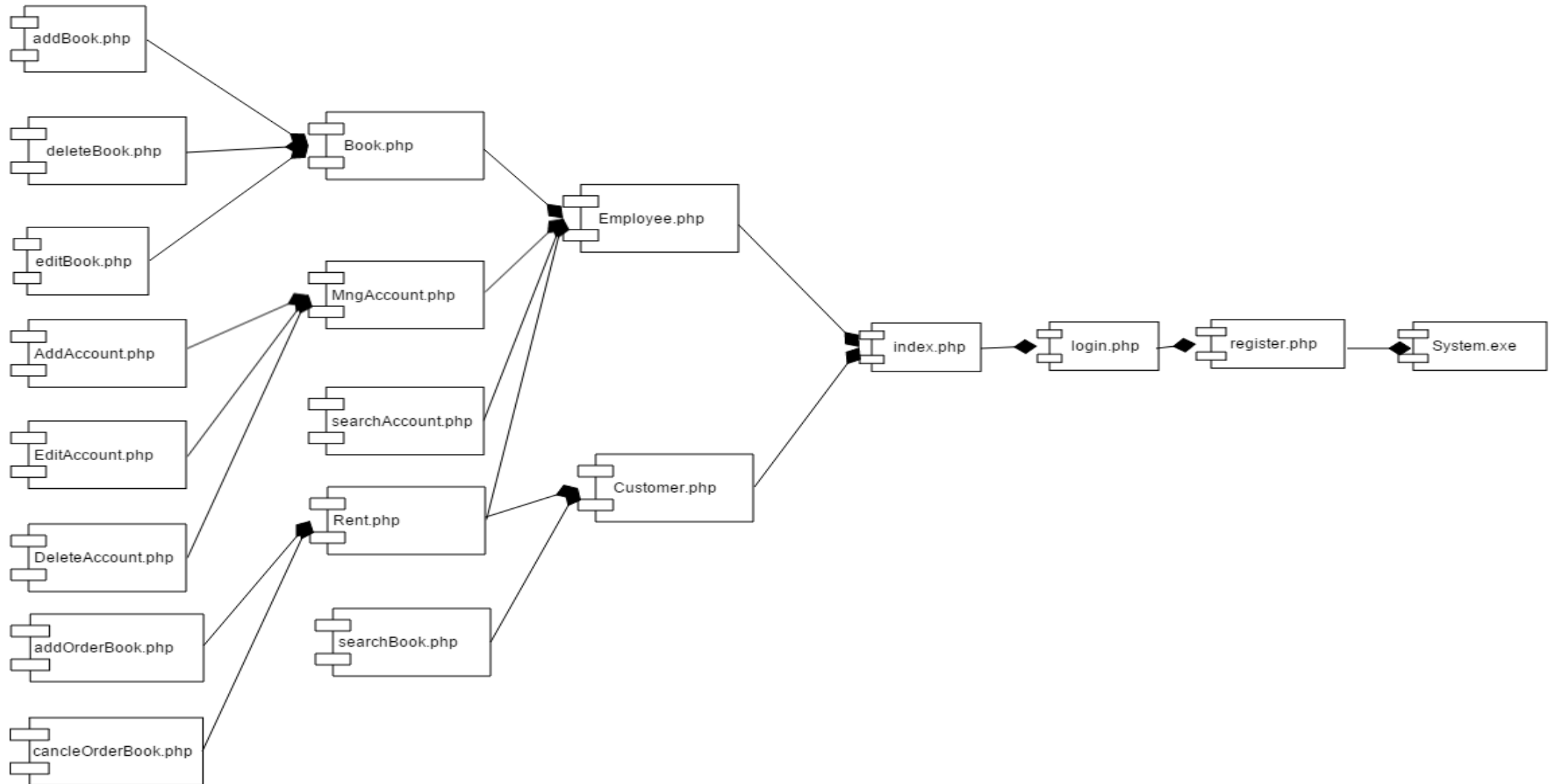


No.	Key	Name	Data Type	Length	Description
1	PK	CUSTOMER_CODE	char	10	รหัสของลูกค้า เช่น C600000001
2		CUSTOMER_NAME	char	30	ชื่อของลูกค้า
3		CUSTOMER_SURNAME	char	30	นามสกุลของลูกค้า
4		CUSTOMER_ADDRESS_NO	char	10	บ้านเลขที่
5		CUSTOMER_ADDRESS_MOO	char	5	หมู่ที่
6		CUSTOMER_ADDRESS_SOI	char	30	ชื่อซอย
7		CUSTOMER_ADDRESS_ROAD	char	30	ชื่อถนน
8		CUSTOMER_ADDRESS_DISTRICT	char	30	ชื่อตำบล
9		CUSTOMER_ADDRESS_AMPHUR	char	30	ชื่ออำเภอ
10		CUSTOMER_ADDRESS_PROVINCE	char	30	ชื่อจังหวัด
11		CUSTOMER_TELEPHONE	char	12	หมายเลขโทรศัพท์
12		CUSTOMER_POINT	long		คะแนนสะสมจากการซื้อสินค้า

ตัวอย่าง Presentation Logic Subsystem



ตัวอย่าง Working Logic Subsystem



Conceptual Model

